

Caleb Binfet

Embedded Diagnostic and Anomaly Detection System Design

1/18/2026

Goal: Design a high-level architecture for a distributed diagnostics, anomaly detection, and self-test framework.

Requirements:

- Collect and transmit telemetry
- Perform lightweight anomaly detection
- Support diagnostics and configuration updates
- Handle temporary outages without losing data
- Reduce memory usage
- Must support none, one, or many sensor interface boards based on a run-time configuration
- Must startup and shutdown safely
- Firmware update process must be secure and fault tolerant

Broad Design Overview:

In my design I chose to prioritize data retention, quick sensor anomaly detection, sensor node modularity and simplicity in the host controller design as my experience matches better with having a complex sensor node than with a complex host controller system.

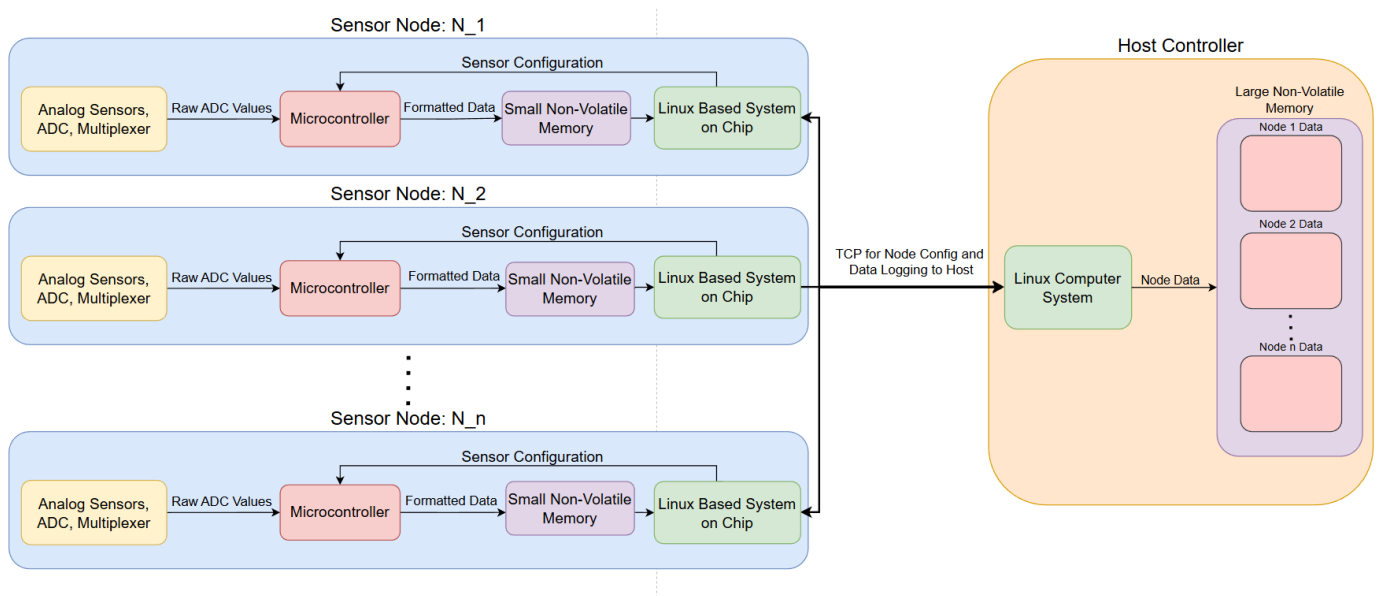


Figure 1: High-level system architecture and communication diagram.

Figure 1 shows the high level system architecture and the basic communication needed to meet the design requirements. The following description will generally flow from left to right on figure 1 as that is also the general direction of flow for the data collected by the monitoring system. A more in depth description of the key system components will follow along with discussion of the design tradeoffs related to each component.

The data is collected via communication between the on-node microcontroller and the analog to digital converters. It is then converted to useful information from the voltage reading based on known sensor constants. The data is formatted and then written to the memory on the node. Then the Linux-based System on chip(SOC) in the sensor node is responsible for reading from the on-node memory, formatting it with the sensor node number, and transmitting the data via a TCP socket back to the host controller. The Host controller organizes it by node number then stores it in the large long-term memory.

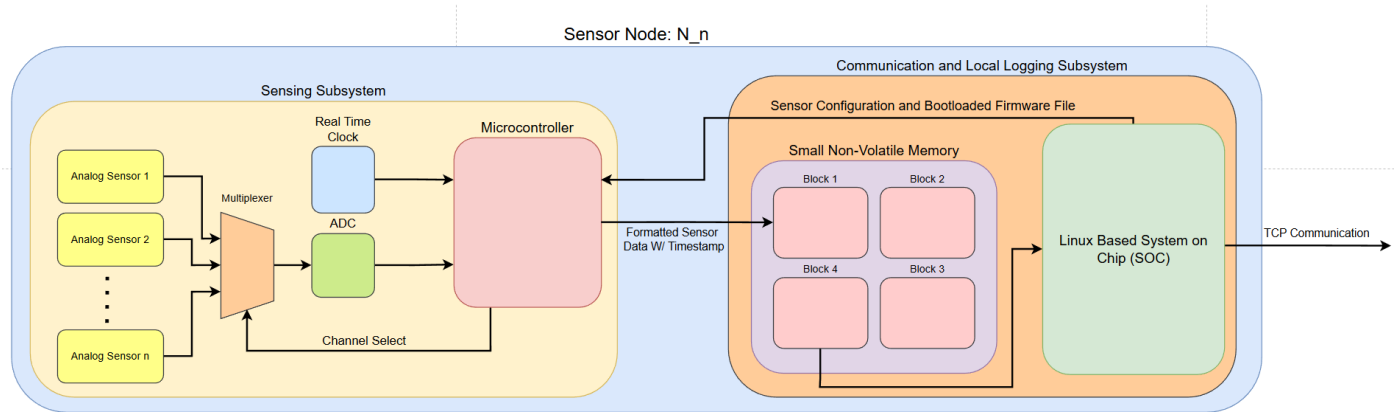


Figure 2: Detailed Sensor Node Architecture and sub-system breakdown.

Figure 2 provides a more detailed picture of the sensor node and how data is handled by the various components.

Sensing Subsystem:

The key advantage of using a microcontroller in the node design is that it allows for constant error checking and system monitoring without having to also manage the communication back to the host. The microcontroller does also make the system relatively easy to change the configuration as more sensors could be added without much additional code. The microcontroller also adds potential for other sensors that are not necessarily analog sensors as it would have the capability to communicate via SPI, I2C,...etc. The microcontroller is also real-time capable and therefore could be responsible for feedback control which is a bonus. Reconfiguring the microcontroller with additional sensors or simply changing the set points for the sensors would be done by communication from the Linux SOC via an on board communication protocol such as UART, SPI, I2C,...etc. The sensing sub-system effectively meets the following requirements: Collect telemetry data, perform lightweight anomaly detection, support diagnostic and configuration updates, supports multiple sensor interfaces based on runtime configuration.

Communication and Local Logging Subsystem:

The communication and local logging subsystem is responsible for reading from the node memory and transmitting the data back to the host as well as receiving configuration updates from the host. The SOC will also act as the bootloader for the microcontroller monitoring system to satisfy the startup and shutdown safety requirement and to make configuration changes and updates relatively simple as the file can be transferred from the host and stored on the SOC.

The memory management is slightly complex but the idea is for the microcontroller to be writing to blocks of memory in a circular manner with the SOC trailing one block behind to avoid read-write

contention issues but, with the right choice of memory, reads and writes can be performed simultaneously. For example, as shown in figure 2, if the microcontroller is writing to block 1 then the SOC would be transmitting the data stored in block 4.

The on node memory allows for data to be retained in the event of a disconnection from the host or sudden power loss at the node as it would be nonvolatile. This does require that the SOC be significantly faster at transmitting than the microcontroller is at writing otherwise data could be lost.

Host and Communication:

The TCP communication will employ the use of a watermark acknowledging system to prevent data loss when logging back to the host. This essentially means that each transmission from the node will include both the memory block number as well as a data line number. The host will respond with the most recent block number and the highest continuous sequence number to make sure that no data lines are lost. For example if the host received lines 101,102,104 it would respond with 102 as that would be the highest continuous number and the node would then know it needs to resend data starting at 103. This satisfies the requirement for temporary outage tolerance.

The Host would continuously cycle in a circular manner between the node sensors and store each nodes data in a separate portion of memory to keep each individual node's data nicely organized. The host would also be responsible for sending adjustments for new sensor limits and send microcontroller configuration files at startup or initiate a stop in sensing for a configuration change. Managing communication would require that this system operates much faster than the nodes or have the node down sample to reduce data sent back to the host for logging.

Security Considerations:

The TCP communication would most likely require the use of TLS to encrypt data in transit as well as some form of authentication by both the host and nodes prior to data transfer. It could also be beneficial to use other similar protocols such as FTPS or HTTPS with higher security.

Key Design Disadvantages:

The node is rather complex and does require a lot of hardware that could be both difficult to implement as well as potentially costly to produce. Determining the right size of on-node memory could be rather difficult to calculate. The full SOC on the node could be overkill but having a full Linux install is most likely more reliable then having the microcontroller perform the task.

Key Design Advantages:

As mentioned above this design does prioritize data retention, node modularity, and host controller simplicity. The design is very modular as adding additional sensors to the microcontroller would be relatively trivial. Splitting the node in to two separate subsystems also has the advantage of allowing for almost entirely parallel and independent development of both node subsystems. The node system has its own non-volatile storage so power loss doesn't result in loss of the most recent sensor data which would be great for potential debugging. Using read-write memory minimizes coordination for writing and reading from the node memory. The communication with the watermark acknowledge reduces memory usage and communication as it does not require constant acknowledgement from the host.